ITA03-MOBILE COMPUTING
ASSIGNMENT-5

ASSIGNMENT 5 – CO5

CO5 Coverage: Six Characteristics of mobile app - connectivity, convenience, localization, reachability, security and personalization - Android Intents and Services - Synchronization and Replication of Mobile Data - Android & IOS multimedia framework (Core Video framework and Core Media framework) -Peer- to Peer Architecture for mobile application -Performance and Multithreading -Graphics and UI Performance.

ANDROID APPLICATION FEATURES & PERFORMANCE ENGINEERING

Course Outcome – CO5: Create mobile application design using diverse mobile operating system frameworks to ensure performance efficiency, security, usability, and scalability.

Nature: Individual

Duration: One Week
Total Marks: 50

Assessment Components

PART A – Design, Analysis & Pseudocode / Architectural Reasoning: 25 Marks

PART B – Implementation, Instrumentation & Computational Investigation: 25 Marks

Submission Date:5/11/2026

PURPOSE OF THE ASSIGNMENT

Modern mobile applications must deliver responsive user experiences while operating under severe constraints of battery, memory, network variability and security threats.

This assignment investigates a fundamental engineering question:

How should a mobile application engineer design, implement and evaluate components so that the application remains efficient, secure, usable and scalable when the number of concurrent users, data volume and device heterogeneity grows?

You are expected to demonstrate the following progression:

Model → Design Intent/Service → Synchronise → Optimise → Measure → Analyse Trade-offs → Approximate / Heuristic → Evaluate

Simply obtaining a working application is not sufficient. Your architectural reasoning, performance instrumentation, experimental evidence and interpretation are all part of the assessment.

PART A – DESIGN, ANALYSIS & ARCHITECTURAL REASONING

25 Marks

All designs and pseudocode must follow a clear, language-independent algorithmic / architectural notation.
Implement the constraint-aware background design from Task A1 (or a closely related sensor-upload scenario).

Your program must record:

● Number of work items enqueued

● Number of work items deferred / cancelled by constraints

● Number of successful uploads / executions

● Battery / network state at each decision point (simulated or real) ●

Maximum concurrent workers

● Final outcome (all data delivered or residual backlog)

B1.1 – Validate the Implementation | 2 Marks

Test the implementation using at least:

● One scenario that succeeds under good connectivity and battery ● One scenario

that must defer or cancel work under Doze / low-battery ● One boundary case

(permission revoked mid-execution)

analyse what each test is intended to verify.

B1.2 – Compare Design with Implementation | 1 Mark

Compare your implementation with your Part A pseudocode. Identify any modification made during implementation and analyse why it was necessary.

B1.3 – Improve the Scheduling Order | 2 Marks

Implement two versions:

Version 1: Process work items in arrival / FIFO order.

Version 2: Use a reasoned priority heuristic intended to surface critical or high-value work earlier while respecting constraints.

You must formulate and analyse your own selection rule.

B1.4 – Measure the Effect | 2 Marks
For the same workloads compare:

Measure	**Version 1**	**Version 2**
Work items enqueued		
Deferred / cancelled		
Successful executions		
Peak concurrent workers		

Did your ordering rule reduce wasted work or improve delivery of critical data? Why or why not?

Evidence Expected: Source code, test suite, instrumentation data, two scheduling strategies and experimental interpretation.

ANSWER – A1

TASK A1 – Design a Constraint-Aware Intent & Service Architecture (7 Marks)

A1.1 – Model the Problem

Element	Description
Input	Sensor readings, timestamps, connectivity state, battery level, Doze state, permission status, user preferences and authentication/logout events.
Output	Encrypted sensor records uploaded to the backend, delivery status and critical-event notifications.
Decision at each stage	Collect or skip a reading; queue locally or upload; select WorkManager or an appropriate foreground service; defer, retry or cancel; notify or suppress notification.
Constraints	Battery and Doze limits, network availability, user permission, authenticated session, encryption, background execution rules and responsiveness.
Goal condition	All eligible records are delivered once or remain safely queued; critical events are notified promptly; no unauthorised data is exposed.

A1.2 – Runtime State

Maintain a state record for each work item: task ID, type, creation time, priority, encrypted payload reference, retry count, network requirement, battery threshold, permission status, authentication status, current state and last error. Global state includes network type/quality, battery level, Doze status, current user/session and maximum permitted concurrency.

A1.3 – Pruning / Feasibility Conditions

- Defer non-urgent uploads when the network is unavailable, metered against user preference, or too weak for the payload.
- Defer non-critical work when battery is below the configured threshold or the device is in Doze; use constraint-aware scheduled work.
- Reject a task if permission is revoked, the user is logged out, the payload cannot be encrypted, or authentication has expired.
- Cancel duplicate work items using a unique task ID; do not delete the only local copy until server acknowledgement is verified.
- Critical alerts may use a permitted urgent notification path, but must still pass authentication, privacy and platform-policy checks.

A1.4 – Architectural Pseudocode

PROCEDURE SCHEDULE_OR_EXECUTE(queue, systemState)\n WHILE queue is not empty DO\n item ← SELECT highest-priority eligible item\n IF item.permissionValid = FALSE OR item.sessionValid = FALSE THEN\n CANCEL item; record reason; CONTINUE\n END IF\n IF item.sensitive AND item.encrypted = FALSE THEN\n REJECT item; record security error; CONTINUE\n END IF\n IF item.requiresNetwork AND systemState.networkAvailable = FALSE THEN\n DEFER item; enqueue WorkManager retry; CONTINUE\n END IF\n IF systemState.doze AND item.priority != CRITICAL THEN\n DEFER item until allowed execution window; CONTINUE\n END IF\n IF systemState.battery < item.minimumBattery AND item.priority != CRITICAL THEN\n DEFER item; CONTINUE\n END IF\n intent ← BUILD explicit Intent for internal component\n worker ← CREATE constrained worker for upload or collection\n result ← EXECUTE worker(item)\n IF result = ACKNOWLEDGED THEN\n MARK delivered; remove local payload after safe acknowledgement\n ELSE IF result = RETRYABLE_FAILURE THEN\n INCREMENT retryCount; retain encrypted local copy; schedule backoff\n ELSE\n CANCEL or quarantine item; log non-sensitive error\n END IF\n END WHILE\nEND PROCEDURE

A1.5 – Trace the Design

Event	Constraint check	Action	Result
1. Sensor reading received	Permission valid; session active	Collect and encrypt reading	Queued safely
2. Network disconnects	Upload requires network	Defer upload; retain local record	No data loss
3. Device enters Doze	Non-critical work restricted	WorkManager waits for eligible window	Battery protected
4. Critical threshold crossed	Permission and notification channel valid	Create high-priority alert and notify through allowed path	User alerted
5. User logs out	Session becomes invalid	Cancel pending authenticated uploads; retain only policy-permitted encrypted records or securely clear them	No upload using stale credentials

Conclusion: The design separates collection, storage, upload and notification. Feasibility checks occur before execution, while retry and cancellation rules preserve privacy and prevent loss of unacknowledged data.

TASK A2 – DON'T SYNCHRONISE WHAT CANNOT BE SAFE

Improve a Weak Data-Replication Design

5 Marks

Consider the following naïve approach for synchronising mobile data with a cloud backend:

ALGORITHM SyncAll(LocalStore, RemoteAPI)

FOR each record R in LocalStore DO

Upload R to RemoteAPI

IF network fails THEN CONTINUE

END FOR

END

This algorithm postpones all decision-making until every record has been examined and ignores conflict, battery and security constraints.

A2.1 – Diagnose the Computational & Security Waste | 1 Mark analyse where

unnecessary work, energy drain and potential data leakage occur.

A2.2 – Redesign Using Incremental & Constraint-Aware Synchronisation | 2 Marks

Write pseudocode that constructs the synchronisation set incrementally. At each

record your algorithm should explicitly represent the alternatives: **Upload →**

Defer → Discard / Resolve Conflict

Introduce appropriate pruning based on network quality, battery level, record priority and encryption status.

A2.3 – Demonstrate Pruning | 1 Mark

Use a concrete set of five records with differing priorities, sizes and sensitivity. Show one branch that your algorithm can terminate early. analyse why further exploration of that branch cannot produce a correct or safe synchronisation.

A2.4 – Challenge Your Pruning Rule | 1 Mark

Suppose offline-first operation with eventual consistency is now required. Determine whether your pruning condition remains valid. If not, identify precisely which assumption has been

violated.

Evidence Expected: Analysis of waste, redesigned pseudocode, pruning trace and reasoning under changed requirements.

ANSWER – A2

TASK A2 – Safe Incremental Data Synchronisation (5 Marks)

A2.1 – Diagnose Waste

The naïve algorithm attempts every record even when the network is unavailable, wastes radio and CPU energy, retries without a policy, ignores priority and battery level, may send unencrypted sensitive records, and does not detect duplicates or conflicts. Continuing after a failed request also hides the difference between a temporary failure and a permanent security error.

A2.2 – Redesigned Pseudocode

PROCEDURE SYNC_INCREMENTALLY(localRecords, state)\n syncSet ← empty\n FOR each record R in priority order DO\n IF R.isDuplicate OR R.expiredByPolicy THEN DISCARD R; CONTINUE\n IF R.sensitive AND NOT R.encrypted THEN QUARANTINE R; CONTINUE\n IF state.permissionInvalid OR state.sessionInvalid THEN STOP safely\n IF state.networkUnavailable OR state.battery < threshold THEN DEFER R; CONTINUE\n IF remoteVersion(R.id) differs from R.baseVersion THEN\n RESOLVE conflict using version/merge policy; CONTINUE\n END IF\n ADD R to syncSet\n response ← UPLOAD R with idempotency key\n IF response acknowledged THEN mark synced\n ELSE IF retryable THEN defer with exponential backoff\n ELSE quarantine and record reason\n END FOR\nEND PROCEDURE

A2.3 – Demonstrate Pruning

Record	Priority / size / sensitivity	Condition	Decision
R1	Critical / 2 KB / sensitive	Encrypted, Wi-Fi available	Upload
R2	Low / 1 MB / non-sensitive	Battery 8%, threshold 15%	Defer
R3	High / 4 KB / sensitive	Encryption missing	Quarantine; do not explore upload branch
R4	Medium / 20 KB / sensitive	Encrypted, network unavailable	Defer
R5	Low / 3 KB / non-sensitive	Duplicate ID already acknowledged	Discard duplicate

Early termination example: R3 is rejected from the upload branch immediately because it contains sensitive data without encryption. Further network retries cannot make this branch safe; encryption or an approved secure representation must be produced first.

A2.4 – Challenge the Rule

Offline-first eventual consistency changes the assumption that records must be synchronised immediately. Deferring a record because the network is unavailable remains valid, but discarding it merely because it is old or temporarily conflicting may violate eventual consistency. The system must retain durable records, use stable IDs and versions, and resolve conflicts when connectivity returns.

TASK A3 – DISCOVER THE MOBILE-BACKGROUND DESIGN PATTERN *5*

Marks

Consider the following three problems:

Background Sensor Collection → Offline Data Synchronisation → Push Notification Delivery

Complete the following design-analysis table:

Design Component	**Sensor Collection**	**Data Sync**	**Push Notification**
State			
Decision variable			
Candidate choices			
Feasibility / Constraint test			
Goal condition			
Pruning / Deferral condition			

Undo / Cancel operation			

A3.1 – Identify the Common Pattern | 2 Marks

Using your completed analysis, analyse what these apparently different mobile problems have in common computationally and architecturally.

A3.2 – Design a Generic Background-Work Framework | 2 Marks Develop

general pseudocode of the form:

SCHEDULE_OR_EXECUTE(State)

Your framework must be sufficiently general to represent all three problems. **A3.3 –**

analyse the Role of Undo / Cancellation | 1 Mark

analyse why a background-work framework must restore or clean up previous state after a cancelled or failed task. Use one of the three problems to illustrate what could happen if this step were omitted.

Evidence Expected: Completed comparison table, generic pseudocode and conceptual explanation.

ANSWER – A3

TASK A3 – Common Mobile Background-Work Pattern (5 Marks)

Design element	Sensor collection	Data synchronisation	Push notification
Component	Sensor API plus foreground service when justified	WorkManager worker / repository	Push messaging service and notification manager
State	Last sample time, permission, sensor status	Queue, retry count, versions, network	Message ID, delivery status, notification permission
Decision variable	Sample now or later	Upload, defer, merge or discard duplicate	Display, defer or suppress
Candidate choices	Collect, batch, pause	Upload, retry, defer, resolve conflict	Notify, group, store silently
Feasibility test	Sensor permission and battery policy	Network, encryption, authentication, battery	Valid payload, app state and notification permission
Goal	Required samples captured	Safe, consistent delivery	Critical information reaches user

Pruning / deferral	Skip redundant samples or batch	Wait for suitable network/battery	Suppress duplicate or invalid notification
Undo / cancel	Stop sensor listener and release resources	Cancel worker; preserve unsynced data	Cancel stale notification or clear pending intent

A3.1 – Common Pattern

All three problems are state-driven background workflows. Each selects a work item, checks constraints, executes or defers it, records outcomes, and handles retries or cancellation. Their common architecture is a scheduler plus state store, feasibility rules, worker, result handler and instrumentation.

A3.2 – Generic Framework

PROCEDURE SCHEDULE_OR_EXECUTE(State)\n update State from OS events\n item ← select eligible item by priority and age\n IF no item exists THEN return WAIT\n IF safety, permission or resource constraints fail THEN defer/cancel item\n ELSE execute item in an appropriate worker\n record start time, resource state and result\n IF success THEN commit state change\n ELSE restore safe state and retry only if permitted\n END IF\n repeat until queue empty or no work is currently feasible

A3.3 – Undo / Cancellation

Cancellation must release resources and restore a consistent state. For example, if a sync worker is cancelled after sending a record but before receiving acknowledgement, it must retain the local record and retry with an idempotency key. Otherwise a record could be lost or duplicated.

TASK A4 – TRACTABILITY: LOCAL vs CLOUD vs HYBRID PROCESSING

Performance, Security and Scalability Classes

4 Marks

Consider the following six mobile-computing problems:

- On-device image classification (tiny model)
- Cloud-based speech recognition
- Peer-to-peer file sharing among nearby devices
- End-to-end encrypted messaging
- Real-time multiplayer game state synchronisation

● Large-scale offline map rendering with vector tiles

For each problem prepare a concise analysis containing:

Problem	**What constitutes a solution?**	**Can a proposed design be efficiently verified?**	**Complexity / Resource class**	**Reason**
1				
2				
3				
4				
5				
6				

Use the resource classes: On-device feasible | Network-bound | Battery-critical | Security-critical | Hybrid required

Inquiry

Why is “runs slowly on a phone” not an adequate definition of a performance problem in mobile computing?

and

Why is the distinction between local processing and cloud offloading important when interpreting scalability and security of a mobile application?

Evidence Expected: Correct classification accompanied by reasoning about verification, resource constraints and problem formulation.

ANSWER – A4

TASK A4 – Local vs Cloud vs Hybrid Processing (4 Marks)

Problem	What constitutes a solution?	Efficiently verified?	Resource class and reason
On-device image classification	Return a class label and confidence for an input image	Yes: compare predictions with labelled test data	On-device feasible; tiny model avoids network delay and protects image privacy.
Cloud speech recognition	Send permitted audio securely and return recognised text	Yes: compare word error rate and latency	Network-bound; quality can be high but depends on connectivity, privacy and server load.
Peer-to-peer file sharing	Transfer verified files between nearby devices	Yes: hashes and transfer logs verify integrity	Hybrid required; local discovery/transfer saves internet use but requires authentication and interruption recovery.
End-to-end encrypted messaging	Only intended endpoints can decrypt messages	Partly: protocol tests verify properties, but formal/security review is also needed	Security-critical; keys, identity verification and secure storage dominate.
Real-time multiplayer synchronisation	Keep player states acceptably consistent within latency bounds	Yes: measure latency, jitter, packet loss and divergence	Hybrid required; local prediction plus server authority balances responsiveness and consistency.
Offline vector map rendering	Render map tiles smoothly without network access	Yes: compare rendered output and frame times	Battery-critical / on-device feasible; caching speeds access but increases storage and memory use.

Why “runs slowly on a phone” is insufficient: performance must be defined with measurable limits such as frame time, latency, memory, energy, throughput and workload. A delay may come from CPU computation, network transfer, disk access, contention or battery throttling.

Why local/cloud distinction matters: local processing avoids network latency and can keep data on the device, but consumes device CPU, memory and battery. Cloud processing can scale server resources, but depends on connectivity, adds transmission cost and exposes data to additional trust boundaries. Hybrid designs select placement per task.

Implement the greedy / adaptive strategy designed in Task A5.

B3.1 – Correctness on Small Instances | 2 Marks

Construct at least three small test instances. For each report Exact cost, Approximate cost, and whether they match.

Do not deliberately choose only instances where the heuristic matches the optimum. **B3.2 –**

Find a Counter-example | 2 Marks

Construct an instance for which your adaptive strategy does not return the minimum cost configuration. Trace the adaptive choices and identify the decision that led away from the optimum.

B3.3 – Increase the Problem Size | 2 Marks

Increase the number of feed items and quality levels. Instrument both approaches. Record meaningful operation counts (configurations evaluated, quality changes, cache evictions) rather than only wall-clock time.

B3.4 – Engineering Trade-off | 2 Marks

Using your experimental evidence, answer: What did the approximate solution gain computationally, and what did it sacrifice in solution quality or guarantee? Identify a realistic situation in which you would accept the approximate solution.

Evidence Expected: Two implementations / simulations, three normal cases, student designed counter-example, scalability evidence and engineering interpretation.
ANSWER – A5

TASK A5 – Approximate UI / Graphics Optimisation (4 Marks)

A5.1 – Exact Perspective
An exhaustive method enumerates every combination of decoding resolution, cache size and prefetch distance, measures estimated frame drops and energy, then chooses the lowest-cost feasible combination. If there are q quality levels, c cache sizes and p prefetch distances, there are q × c × p configurations per device profile. Testing many profiles and feed states quickly becomes expensive and can delay adaptation.

A5.2 – Adaptive Heuristic

PROCEDURE ADAPT_FEED(metrics, config, battery)\n REPEAT\n measure frameTime, droppedFrames, memoryUse and battery\n IF frameTime > 16 ms OR memoryUse > limit THEN\n reduce decode quality one step; evict least-recently-used off-screen item\n shorten prefetch window\n ELSE IF frameTime < 12 ms AND battery is healthy AND memory has headroom THEN\n increase quality one step or extend prefetch

window\n END IF\n avoid changing more than one major parameter per evaluation\n update config and record the reason for each change\n UNTIL frameTime remains within target for several windows OR no safe adjustment remains\n RETURN config

A5.3 – Trade-off

A heuristic can find a feasible smooth-scrolling configuration quickly, but it does not prove that the total energy cost is globally minimal for a given visual quality. It may settle on a local choice or react to short-term measurements. It is acceptable for real-time feeds because responsiveness matters and conditions change continuously; periodic measurement and safe limits reduce risk.

PART B – IMPLEMENTATION & COMPUTATIONAL INVESTIGATION

The following compact simulations provide reproducible example logic and illustrative tables. Run equivalent code on your chosen device/emulator and replace illustrative values with measured evidence for final submission.

Use your background-work implementation from Task B1.

Construct two workloads having the same number of records / events and using the same constraint thresholds.

Workload A – Design an instance that your algorithm resolves with relatively little deferral or cancellation.

Workload B – Design an instance intended to cause substantially more deferrals, retries or constraint violations.

Before Running – Predict which workload will require more work, where you expect unsuccessful attempts to arise, and why pruning / deferral is likely to occur earlier or later.

Then execute both and compare: work items processed, deferrals, successful deliveries.

Investigation – analyse why input size alone fails to describe the actual computational and energy effort required by a mobile background pipeline on a particular instance. Identify the structural property of your difficult workload that caused additional work.

Evidence Expected: Two student-designed workloads, prediction, measured behaviour and interpretation.

ANSWER – B1

TASK B1 – Instrumented Background Architecture (7 Marks)

B1.1 – Implementation Model and Tests

Each work item has an ID, priority, encrypted flag, network requirement and status. The scheduler logs enqueue, defer, cancel and success events. A unique ID prevents duplicate

scheduling. Workers are bounded to a small concurrency limit, and retryable failures use backoff.

Test	Input condition	Expected behaviour	What it verifies
Good connectivity	Battery 80%, Wi-Fi available, permission valid	Eligible encrypted items upload and are acknowledged	Normal successful execution
Doze / low battery	Battery 10%, non-critical task, Doze active	Task is deferred and retained	Constraint-aware scheduling and battery policy
Permission revoked	Permission becomes invalid during worker	Worker stops safely; no new sensor read/upload	Boundary handling and privacy

B1.2 – Design vs Implementation

The pseudocode describes the architecture independently of language. In implementation, the OS may stop a worker or process unexpectedly, so state must be persisted before execution and each upload must be idempotent. These changes make retries safe after process death.

B1.3 – Two Scheduling Strategies

Version 1 (FIFO) selects the oldest queued task. Version 2 uses a score: score = 5 × priority + 2 × ageBonus + criticalFlag, subject to permission, encryption, network and battery constraints. Critical work receives an earlier opportunity, but unsafe items are never promoted past a security check.

B1.4 – Illustrative Measurement

Measure	Version 1: FIFO	Version 2: priority heuristic
Work items enqueued	10	10
Deferred / cancelled	4	4
Successful executions	6	6
Peak concurrent workers	2	2
Critical records delivered first	2 of 4	4 of 4

Interpretation: Both strategies obey the same constraints and complete the same number of eligible items. The priority strategy improves delivery order for critical records, but it does not reduce the number deferred in this example. Different workloads may produce different results.

TASK B2 – ENGINEER AN ADVERSARIAL WORKLOAD

Same Size, Very Different Behaviour

5 Marks
ANSWER – B2

TASK B2 – Adversarial Workload (5 Marks)

Property	Workload A: easy	Workload B: difficult
Number of records	8	8
Network	Stable Wi-Fi	Repeated disconnects / weak signal
Battery	70–90%	8–18%
Security state	All sensitive records encrypted	Some records unencrypted or expired session
Priority mix	Few high-priority records	Several critical records plus low-priority large records
Prediction	Most eligible records succeed in first attempt	More deferrals, retries and quarantines

Illustrative result	Workload A	Workload B
Items processed	8	8
Deferrals	1	5
Successful deliveries	7	3
Security quarantines	0	2

Input size alone is not a reliable measure of computational or energy effort. Workload B has the same record count but more network failures, constraint checks, retries and security rejections. Its structure—not just size—causes additional processing and radio wake-ups.

TASK B3 – EXACT vs APPROXIMATE UI / GRAPHICS OPTIMISATION *8*

Marks

ANSWER – B3

TASK B3 – Exact vs Approximate UI Optimisation (8 Marks)

Solution A: Exact Search
For a small instance, enumerate each quality/cache/prefetch configuration, calculate cost = 4 × droppedFrames + energyUnits + 2 × memoryPenalty, reject configurations exceeding the frame or memory limit, and return the feasible configuration with minimum cost.

Solution B: Adaptive Strategy
Start with medium quality, moderate cache and short prefetch. If frame time exceeds 16 ms, reduce quality and evict the least-recently-used item. If frame time is comfortably below target and battery is healthy, cautiously increase quality or prefetch.

B3.1 – Three Small Test Instances

Instance	Exact minimum cost	Approximate cost	Match?
1: steady Wi-Fi, moderate feed	18	18	Yes
2: low-memory device	12	14	No
3: flagship, stable frame time	10	10	Yes

These are illustrative simulation values, not device measurements. The table deliberately includes a mismatch to avoid assuming the heuristic always matches the optimum.

B3.2 – Counter-example
Suppose high quality with a small cache has total cost 9, while medium quality with a larger cache has cost 11. The adaptive strategy observes one brief frame-time spike, reduces quality, and then increases cache size to avoid future misses. Its local reaction selects cost 11 and does not return to the globally better configuration. The decision is locally reasonable but misses the interaction between quality and cache.

B3.3 – Increasing Problem Size

Problem size	Exact configurations evaluated	Adaptive quality changes	Adaptive cache evictions
3 items, 2 quality levels	8	2	1
10 items, 4 quality levels	64	4	5
50 items, 6 quality levels	216	6	17

The counts are illustrative and should be regenerated from the submitted implementation. Exact search evaluates every configuration; adaptive search evaluates a small number of changes based on current measurements.

B3.4 – Engineering Trade-off
Approximation reduces search work and responds quickly to changing frame time, but it sacrifices a global optimality guarantee and can choose a higher-cost configuration. It is suitable

for continuous scrolling, where a slightly suboptimal configuration is preferable to visible stalls while exhaustive search runs.

TASK B4 – FINAL ENGINEERING DECISION

Multimedia Framework Selection under Production Constraints *5 Marks*

A production mobile application must capture, encode and upload short video clips from thousands of concurrent users. Each device has different camera hardware, CPU, battery state and network conditions. A placement / encoding decision is required quickly.

B4.1 – Implement a Practical Strategy | 2 Marks

Implement a reasonable heuristic strategy for choosing resolution, bitrate and upload timing. State the selection rule used. Output must show chosen parameters, estimated energy / time and remaining battery impact for a set of sample devices.

B4.2 – Challenge Your Strategy | 1 Mark

Construct an input (device profile + network) intended to make your strategy choose a suboptimal configuration, or analyse through comparison with a small exhaustive search why the heuristic may fail to be optimal.

B4.3 – Production Decision Note | 2 Marks

Prepare a short Algorithm / Architecture Engineering Decision Note addressing:

● Why exhaustive search over encoding parameters is unattractive at production scale

● What guarantee your chosen strategy does or does not provide ● What

computational / energy advantage it provides

● When an exact or cloud-assisted approach might still be appropriate ● What

evidence you would monitor before deploying the strategy

Evidence Expected: Working implementation or detailed simulation, allocation / parameter output, challenging input and evidence-based engineering decision.

MARK DISTRIBUTION

Component	**Task**	**Marks**
Part A – Design & Reasoning	A1 – Intent/Service Architecture	7
	A2 – Sync & Pruning	5
	A3 – Background Design Pattern	5
	A4 – Resource Classes	4
	A5 – UI Approximation Design	4
Part A Total		**25**
Part B – Implementation	B1 – Instrumented Background	7
	B2 – Adversarial Workload	5
	B3 – Exact vs Approximate UI	8
	B4 – Multimedia Decision	5
Part B Total		**25**
TOTAL		**50**

SUBMISSION REQUIREMENTS

For Part A, submit the required problem formulation, pseudocode / architectural diagrams, traces, comparison tables and reasoning. Programming-language code will not be accepted as pseudocode.

For Part B, submit source code together with test inputs, expected behaviour, actual outputs, instrumentation data and interpretation. A screenshot showing only a final answer is not sufficient evidence.

Students are expected to use the same engineering workflow throughout the assignment:

UNDERSTAND → DESIGN → CONSTRAINT-CHECK → IMPLEMENT → MEASURE → CHALLENGE → INTERPRET

The key question is not simply "Did the application work?" It is also:

How much background work, energy and network traffic was required, why was it required, when does the design stop being practical, and what should an engineer do about it?

ANSWER – B4

TASK B4 – Final Engineering Decision: Multimedia Framework (5 Marks)

B4.1 – Practical Encoding Heuristic

Selection rule: if battery is below 20% or network is weak, choose 480p at low bitrate and defer upload until a stable connection; if battery is at least 40% and Wi-Fi is stable, choose 1080p at medium/high bitrate; otherwise choose 720p at medium bitrate. Always encrypt the clip before queueing and upload only after authentication.

Device profile	Battery / network	Chosen settings	Upload decision	Illustrative estimate
A	80%, Wi-Fi	1080p, 4 Mbps	Upload now	2 min; 3% battery
B	35%, LTE fair	720p, 2 Mbps	Upload when stable	3 min; 4% battery
C	15%, weak LTE	480p, 0.8 Mbps	Defer until Wi-Fi/charging	1 min encode; 1% battery
D	55%, Wi-Fi, hot device	720p, 1.5 Mbps	Wait for temperature to fall	2 min; 2% battery

B4.2 – Challenge the Strategy

A device with 18% battery but an urgent emergency clip over excellent Wi-Fi may be assigned 480p and deferred by the heuristic, even though a higher-quality immediate upload may be required. This is a counter-example because the rule values battery over urgency. A production system should include a user/business priority flag and an emergency override with explicit consent and energy limits.

B4.3 – Production Decision Note

- Exhaustive search over resolution, bitrate, codec, upload timing and device profiles is expensive at thousands of concurrent users and may not finish before the decision is needed.
- The heuristic guarantees that selected settings follow stated thresholds, but it does not guarantee globally minimum energy or maximum quality.
- It reduces computation by evaluating a small number of rules and reduces energy through lower bitrate, batching and deferred uploads.
- Exact or cloud-assisted optimisation may be appropriate for offline planning, high-value clips, or devices with sufficient resources and stable connectivity; urgent cases may use a priority-aware policy.
- Before deployment monitor encode time, upload latency, failure/retry count, battery drain, temperature, video quality, privacy/security errors and completion rate.

OVERALL CONCLUSION

The assignment demonstrates that mobile applications need constraint-aware scheduling, secure incremental synchronisation, explicit background-work state, and measurable UI/multimedia optimisation. Exact search can provide an optimum for small spaces, while adaptive heuristics reduce decision cost for dynamic mobile conditions. A reliable design must preserve encrypted data during failures, respect permissions and OS limits, instrument real behaviour, test adverse cases, and clearly state where a heuristic sacrifices optimality.